

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 6

Ejemplo integrador: conversor de unidades

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles**SEMESTRE** 6.º · Mención Ing. de Software**TEMA** Ejemplo resuelto de consolidación**DURACIÓN** 1 sesión**CAPÍTULOS** Consolida Cap. 1–3 (Guías 1 a 5)**MODALIDAD** Laboratorio · individual**DOCENTE** M. Sc. Lic. Huáscar Fedor Gonzales Guzmán**REQUISITO** Guías de Laboratorio N.º 1 a N.º 5**NOTA · Un ejemplo que reúne lo aprendido**

Esta guía no introduce temas nuevos: es un **ejemplo integrador resuelto** que combina layout, entrada de texto, un `DropDownButton` y `setState`. Construirás un **conversor de unidades** que calcula el resultado en vivo mientras el usuario escribe.

01 Objetivos

Objetivo general. Integrar los conocimientos de las Guías 1 a 5 construyendo una aplicación de conversión de unidades con entrada de datos, selección de categoría y cálculo reactivo.

Objetivos específicos:

- Capturar un valor numérico con `TextField` y `TextEditingController`.
- Seleccionar la conversión con un `DropDownButton`.
- Recalcular el resultado en vivo con `setState` y `onChanged`.
- Organizar la interfaz con `Column`, `Padding` y `Card`.
- Aplicar una arquitectura simple separando datos (categorías) de la interfaz.

02 Análisis del problema

La app tendrá una sola pantalla con: un campo para el valor a convertir, un menú para elegir el tipo de conversión y un resultado que se actualiza automáticamente. Soportaremos tres conversiones:

EJEMPLO · Conversiones soportadas**Bolivianos** → **Dólares**: dividir entre 6,96.**Celsius** → **Fahrenheit**: multiplicar por 9/5 y sumar 32.**Kilómetros** → **Millas**: multiplicar por 0,621371.**NOTA · Separar datos de interfaz**

En lugar de llenar el código con condicionales, describiremos cada conversión como un **dato** (nombre + fórmula). Así, agregar una nueva conversión será añadir un elemento a una lista, no reescribir la lógica.

03 Requisitos previos

Requisito indispensable. Haber completado las Guías N.º 1 a N.º 5.

Conceptos que se reutilizan:

StatefulWidget + setState

TextField + controller

DropDownButton

Column / Padding

Card

Funciones en Dart

04 Desarrollo · Parte A — Modelar las conversiones

1 Definir el modelo de conversión

1. Crea el proyecto `converter_app`. En `lib/main.dart`, debajo de los imports, define una clase que guarde el nombre de la conversión y su fórmula como una **función**:

```
LIB/MAIN.DART · DART
class Conversion {
  final String nombre;
  final String unidadEntrada;
  final String unidadSalida;
  final double Function(double) formula;

  const Conversion(
    this.nombre,
    this.unidadEntrada,
    this.unidadSalida,
    this.formula,
  );
}
```

DEFINICIÓN · Una función como dato

`double Function(double)` es el tipo de una función que recibe un `double` y devuelve otro. Guardar la fórmula así permite aplicar la conversión con `conversion.formula(valor)`, sin condicionales.

2 Crear la lista de conversiones

1. Debajo de la clase, declara la lista con las tres conversiones:

```
LIB/MAIN.DART · DART
const factorBob = 6.96;
```

```
final conversiones = <Conversion>[
  Conversion('Bolivianos a Dolares', 'Bs', 'USD',
    (v) => v / factorBob),
  Conversion('Celsius a Fahrenheit', 'C', 'F',
    (v) => v * 9 / 5 + 32),
  Conversion('Kilometros a Millas', 'km', 'mi',
    (v) => v * 0.621371),
];
```

05 Desarrollo · Parte B — Construir la interfaz

3 Preparar la pantalla con estado

1. Crea un `StatefulWidget`. Guarda el controlador del campo, la conversión seleccionada y el resultado:

```
LIB/MAIN.DART · DART
class PantallaConversor extends StatefulWidget {
  const PantallaConversor({super.key});

  @override
  State<PantallaConversor> createState() =>
    _PantallaConversorState();
}

class _PantallaConversorState
  extends State<PantallaConversor> {
  final _controller = TextEditingController();
  Conversion _seleccion = conversiones.first;
  double _resultado = 0;

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
}
```

4 La lógica de conversión

1. Agrega un método que lea el texto, aplique la fórmula y actualice el resultado con `setState`:

```
LIB/MAIN.DART · DART
void _convertir() {
  final valor = double.tryParse(_controller.text);
  setState(() {
    _resultado =
      valor == null ? 0 : _seleccion.formula(valor);
  });
}
```

```
});
}
```

ADVERTENCIA · `double.tryParse` en vez de `parse`

`double.tryParse` devuelve `null` si el texto no es un número, en lugar de lanzar un error. Así la app no se rompe cuando el campo está vacío o mal escrito.

5 Construir el build: campo, menú y resultado

1. Arma la interfaz con una `Column`. El campo llama a `_convertir` en cada cambio; el menú actualiza la selección y recalcula:

```
LIB/MAIN.DART · DART
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Conversor')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          TextField(
            controller: _controller,
            keyboardType: TextInputType.number,
            decoration: const InputDecoration(
              labelText: 'Valor a convertir',
              border: OutlineInputBorder(),
            ),
            onChanged: (_) => _convertir(),
          ),
          const SizedBox(height: 16),
          DropdownButton<Conversion>(
            isExpanded: true,
            value: _seleccion,
            items: conversiones.map((c) {
              return DropdownMenuItem(
                value: c,
                child: Text(c.nombre),
              );
            }).toList(),
            onChanged: (nueva) {
              setState(() => _seleccion = nueva!);
              _convertir();
            },
          ),
        ],
      ),
    ),
  );
}
```

}

6 Mostrar el resultado en una Card

1. Dentro de la `Column`, debajo del menú, añade una `Card` que muestre el resultado formateado:

DART · DENTRO DE LA COLUMN

```
const SizedBox(height: 24),
Card(
  child: Padding(
    padding: const EdgeInsets.all(20),
    child: Text(
      _resultado.toStringAsFixed(2) + ' ' +
        _seleccion.unidadSalida,
      style: const TextStyle(
        fontSize: 28,
        fontWeight: FontWeight.bold,
      ),
    ),
  ),
),
),
),
```

2. Ejecuta: escribe un número y cambia la conversión en el menú; el resultado se actualiza al instante.

06 Resultado esperado

Al terminar tendrás una pantalla con el campo de entrada, el menú de conversiones y una tarjeta con el resultado en vivo. Ejemplos de comportamiento:

Conversión	Entrada	Resultado
Bolivianos a Dólares	100	14.37 USD
Celsius a Fahrenheit	37	98.60 F
Kilómetros a Millas	10	6.21 mi

07 Verificación de resultados y entregable

Marca cada punto como verificación de que completaste el ejemplo:

- ☐ Las tres conversiones están modeladas como datos (nombre + fórmula).
- ☐ El campo acepta números y usa `double.tryParse` sin romperse.
- ☐ El `DropDownButton` cambia la conversión y recalcula.
- ☐ El resultado se actualiza en vivo al escribir (`onChanged`).
- ☐ El resultado aparece en una `Card` con dos decimales y su unidad.

EJEMPLO · Qué entregar

El archivo `lib/main.dart` terminado y un documento (PDF o Word) con: (1) captura de una conversión de moneda; (2) captura de una conversión de temperatura; y (3) **reto personal**: agrega al menos dos conversiones más (por ejemplo, `kg` → `lb` y `m` → `pies`). Recuerda: solo debes añadir elementos a la lista `conversiones`, sin tocar la interfaz. Descríbelo en 3–5 líneas.

08 Cuestionario

1. ¿Qué ventaja tiene guardar la fórmula como una función dentro del modelo, en vez de usar condicionales?
2. ¿Por qué se usa `double.tryParse` en lugar de `double.parse`?
3. ¿Qué provoca la actualización del resultado mientras el usuario escribe?
4. ¿Qué función cumple `isExpanded` en el `DropDownButton`?
5. Si quisieras agregar una nueva conversión, ¿qué parte del código tocarías y por qué?

09 Rúbrica de evaluación

Criterio	Puntaje
Modelo de conversiones (datos + fórmula)	25 %
Entrada de datos y cálculo reactivo	30 %
<code>DropDownButton</code> y resultado en Card	20 %
Reto personal (nuevas conversiones)	10 %
Cuestionario resuelto	15 %

10 Referencias

- Flutter — Create and style a text field. docs.flutter.dev/cookbook/forms/text-input
- Flutter — `DropDownButton` (API). [api.flutter.dev · DropdownButton](https://api.flutter.dev/DropdownButton)
- Flutter — Basic widgets (Column, Padding, Card). docs.flutter.dev/ui/widgets/layout

Ejemplo integrador de las Guías 1–5. Contenido basado en la documentación oficial de Flutter (v3.44). Elaborado para INF 662 — UATF.